

CS422 Lab3: Socket programming and Network Address Translation

Due: **23:59:59 PM, Mon Nov 10, 2025**

Total points: **60 points**

1 Goal

The goal of this lab is to understand basic socket programming and to implement your own server and client programs. Afterwards, you will extend your socket programs to mimic how Network Address Translation (NAT) works.

2 Before you start

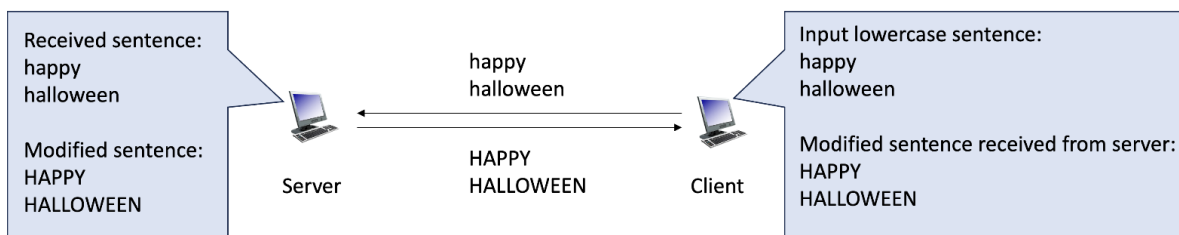
1. In this lab, you will need to implement your own server and client programs using socket programming. The sample codes are provided along with this assignment (**client.c**, **server.c** and **nat.c** files) and you are required to complete the **TODOs** inside them.
2. For this lab, You should first test your code on your own computer. Once you confirm that everything works correctly, you can then run the programs on the lab machines in **HAAS G050**.
3. Learn how to find the IP address of your current machine. (**ipconfig** for windows, **ifconfig** for linux/macos)
4. You must work individually on this assignment. And please start early.

3 Part I: Build Client-Server over Socket Programming (25 points)

3.1 Code implementation (20 points)

In this part, you will set up a server and a client using TCP socket programming. A tutorial on socket programming is provided in the lecture slides on the course homepage. Sample code files are also provided to help you get started.

Basically, the server opens a server process and the client creates a client process to connect to this server process. After starting the server process, you then run the client process, and you will be prompted to input a message on the terminal, and this message will be sent to the server after you pressing **Ctrl + D**. Upon receiving the message, the server will process the message and send it back to the client. The client will then display it on its terminal. The server will also display the message on its terminal:



- You need to revise the provided codes to allow the client and server to run on different machines which can be specified by the ip address and port number in the command line. You should run the code using the following commands:

```
./server [port#]
./client [ip_server_addr] [server_port#]
```

Hints: When running both the client process and server process on your laptop, please set the `ip_server_addr` to 127.0.0.1. Later, when running the two processes on different lab machines, you need to find the current ip address of each machine. (`ipconfig` for windows, `ifconfig` for linux/macOS)
- We expect that the client allows you to input multiple lines. Each message should end with `\n`. Please use keyboard shortcut **Ctrl + D** to signal that you have finished typing all your input messages. The server should CAPITALIZE the received input message and print both the original message and capitalized message on its terminal, and send back the capitalized messages to the client. On the client side, the capitalized messages will be printed on the terminal when it is received by the client.
- There are **3 TODOs** in the `server.c` file and **3 TODOs** in the `client.c` file.

3.2 Answer questions (5 points)

- (1 point) From the code, how can you tell this socket is using TCP?
- (2 points) Please show the ip addresses of the two lab machines you used (for example, `amber01@cs.purdue.edu` and `amber05@cs.purdue.edu`). Are they private ip addresses or public ip addresses?
- (2 points) When you tested your code across two lab machines, did they communicate successfully using their ip addresses and port numbers? What would happen if you run **one of the processes** on your home computer without using Purdue VPN? Can they still communicate?

4 Part 2: Build NAT over Socket Programming (35 points)

4.1 Code implementation (25 points)

Now you are required to use socket programming to mimic how NAT works. Assume the client now is running on a private host machine behind the NAT server. And NAT is running on a third machine. You will **reuse** the previous `server.c` and `client.c` files without any modifications. Please modify the provided `nat.c` file to meet the following requirements:

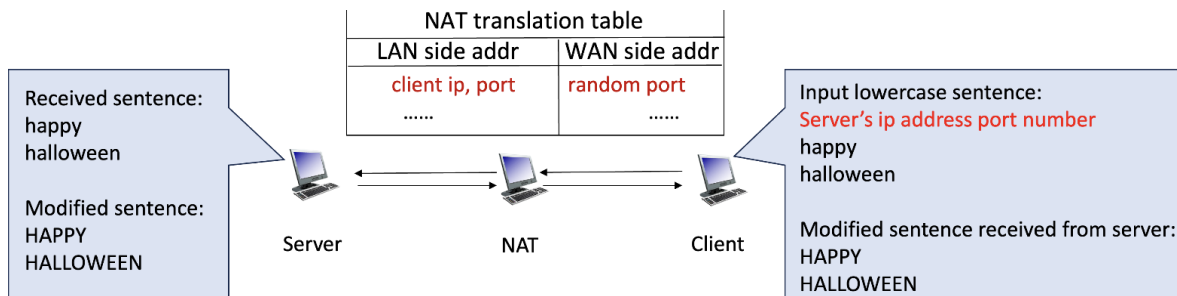


Figure 1: The overview of NAT.

- When entering messages on the client side, please use the first line to specify the IP address and port number of the “public server”, in the following format: `[ip_server_addr] [server_port#]`. For the remaining lines, enter the sentences you want the server to capitalize for you (see Figure 1).

The server process, NAT process and the client process should run the following commands (please make sure your first run the server process, then NAT process and finally the client process):

```
./server [port#]
./nat [nat_port#]
./client [ip_nat_addr] [nat_port#]
```

- Once the NAT server process receives a message from the private client, it will create a new child process to handle this specific connection.

Hints: NAT process uses `fork()` to handle multiple operating-system processes. (Note: Here, process refers to an **OS-level** process, which is different from the application-layer process.) The parent process keeps listening for incoming connections, and each time a new connection is established, it forks a child process to handle that connection. The main structure of this logic has already been implemented for you. You only need to understand how it works. :)

- Inside the child process, perform the following steps:
 - Step 1: Parse the received message from the private client. Extract the server's ip address and port number from the first line, and keep the remaining lines as the actual message that will later be sent to the server (**TODO-1**). Store the private client's ip address and port number (**TODO-2**).
 - Step 2: Open a new socket with a specific port number. Note that, the NAT server will maintain a table named `nat_table.txt`, where each line records the following information: `[client_ip] [client_port#] [nat_port#]`. You must ensure that the assigned NAT port number does not overlap with any existing entry in the table (**TODO-3**). Once a valid port number is selected and the socket is successfully created, add a new entry to the table for the current client (**TODO-4**).
Hints: When assigning a new port number, we always begin the search from 40000. Therefore, for simplicity, set your `[nat_port#]` **smaller than 40000**, making sure it will not overlap with it.
 - Step 3: Forward the client's actual message to the public server.
 - Step 4: After receiving the modified message (in uppercase) from the server, send it back to the client, and close this socket.
- After sending the modified message to the client, terminate the child process. The parent process continues listening for new connections and repeats this procedure whenever another message arrives.
- There are **4 TODOs** in the `nat.c` file.

4.2 Answer questions (10 points)

- (3 points) Run the server process on one machine and the NAT process on another machine. Then, run two client processes on two different machines (both communicating with the same server). Please include screenshots of all four terminals and a screenshot of the `nat_table.txt` file. (see `sample_output.txt`)
- (1 point) Suppose the NAT process is running on a machine with ip address `ipA`, and a client is running on a machine with ip address `ipB`. When the server eventually receives a message from the client, what is the source IP address shown at the server side?
- (2 points) List at least two advantages of using NAT.
- (2 points) In practice, how does a newly joined private client discover the ip address of the router that is running NAT?
- (2 points) When two different private clients send messages to the same public server, how does the NAT device distinguish between their connections?

5 Materials to turn in

You will submit your assignment on **gradescope**. You submission should zip all the files into “lab3_UID*.zip”, including the following files

- **Program files:** client.c, server.c and nat.c.
- **Report (pdf or word):** First, please include your name and UID on the cover page. In the report, Please answer the above questions and paste the required printouts in the right place.

6 Support

Questions about the assignment should be posted on **Campuswire** or asked during **PSOs**.